

Chihuahua vs Muffin — Data-Centric AI Approach

3LC HACK[AI]THON 2.0 — Problem-Solving & Dataset Improvement Report

1. Problem Understanding

The task was to classify images into two classes: Chihuahua and Muffin. The hackathon followed a data-centric AI approach, so the main focus was improving the quality of the training data rather than changing the prescribed model.

2. Initial Dataset and Baseline

The initial training table contained 3,733 images: 50 Chihuahua images, 50 Muffin images, and 3,633 images marked undefined with weight 0. The validation set contained 1,000 labeled images (500 Chihuahua and 500 Muffin).

The supplied pipeline used a ResNet-18 with random initialization, following the competition rules. The baseline training run reached a best validation accuracy of 75.60%.

3. Tools Used

- Python 3.11 and the supplied 3LC starter kit.
- PyTorch with the provided ResNet-18 training pipeline.
- 3LC Dashboard for dataset inspection, filtering, metrics, label editing, weights, and embeddings.
- UMAP embeddings for visual analysis of the training examples.
- Kaggle for hidden-test evaluation of submission.csv.

4. Data-Centric Investigation

After establishing the baseline, the model outputs were examined at the individual-example level. The 3LC metrics included loss, predicted label, confidence, accuracy, weight, and embedding coordinates.

- High-loss and uncertain examples were considered candidates for inspection.
- UMAP visualization was used to understand the distribution of examples in embedding space.
- Images were inspected before assigning a new label.
- Clearly identifiable Chihuahua images were labeled Chihuahua and given weight 1.
- Clearly identifiable Muffin images were labeled Muffin and given weight 1.
- Ambiguous or unidentifiable examples were kept undefined with weight 0 instead of forcing a potentially incorrect label.

5. Dataset Editing

The main improvement was targeted labeling of useful examples from the previously undefined portion of the training set. Changes were made in 3LC and committed before retraining so the updated table became the training source.

6. Retraining and Validation

The same model architecture and training pipeline were retained so that changes in performance could be evaluated primarily as a result of dataset improvements.

- Epoch 1: 53.80%
- Epoch 2: 59.60%

- Epoch 3: 65.50%
- Epoch 4: 61.30%
- Epoch 5: 71.60%
- Epoch 6: 73.50%
- Epoch 7: 73.80%
- Epoch 8: 74.90%
- Epoch 9: 75.20%
- Epoch 10: 75.60% — best validation accuracy

7. Test Prediction and Submission

The supplied prediction script was used with `best_model.pth`. It predicted all 1,184 test images, aligned the predictions to `sample_submission.csv`, and generated `submission.csv`. The file was uploaded to Kaggle.

8. Kaggle Result

Best reported Kaggle public score: 0.88006

The uploaded Kaggle leaderboard screenshot confirms a score of 0.88006. Kaggle provided the external evaluation signal on the hidden test set, which was used to evaluate generalization.

9. Key Insights

- Data quality can be more important than model changes in a data-centric competition.
- Incorrect labels can introduce training noise, so ambiguous images should not be forced into a class.
- Loss and confidence are useful signals for finding examples that deserve human inspection.
- Embedding visualization can reveal groups and unusual examples that are difficult to identify through random inspection.
- Dataset edits should be committed and followed by retraining and evaluation.
- Validation accuracy and Kaggle score measure different things: validation uses the supplied validation set, while Kaggle evaluates the hidden test set.

10. Final Workflow

1. Register the dataset in 3LC.
2. Train the baseline model and record validation accuracy.
3. Generate per-example metrics and UMAP embeddings.
4. Find difficult, high-loss, or uncertain examples.
5. Inspect those images and correct only confident labels.
6. Set reliable labeled examples to weight 1 and leave ambiguous examples at weight 0.
7. Commit the edits in 3LC.
8. Retrain the same model.
9. Generate test predictions.
10. Create `submission.csv`.
11. Upload the submission to Kaggle and compare the score.

12. Repeat the data-improvement cycle when time permits.

11. Conclusion

The solution followed the intended data-centric workflow: establish a baseline, use model metrics and embeddings to locate potentially problematic examples, improve the dataset through targeted human inspection, retrain using the prescribed model setup, and evaluate through Kaggle. The best reported public Kaggle score was 0.88006.